

# Projet Processeur Monocycle et Multicycle

## Un compte-rendu global est demandé pour ce projet

L'objectif de ce projet est de concevoir et de simuler l'architecture interne d'un processeur. Celui-ci sera construit à partir de composants élémentaires tels que des registres, des multiplexeurs, des bancs de mémoire et une Unité Arithmétique et Logique (UAL). Ces différents composants seront assemblés afin de former les principaux blocs du système : l'unité de traitement, l'unité de gestion des instructions et l'unité de contrôle. Le fonctionnement du processeur sera d'abord validé par simulation grâce à l'exécution d'un programme de test simple, puis implémenté et testé sur un FPGA.

Pour chaque conception, les circuits seront décrits en VHDL comportemental et simulés à l'aide de Modelsim (ou GHDL/GTKwave), en utilisant un banc de test spécialement développé pour cela.

Compétences :

- **Rendre compte de votre travail par la rédaction d'un compte rendu de projet**
- **Concevoir, synthétiser et valider une IP numérique décrite en VHDL RTL**

## PARTIE 1 – UNITE DE TRAITEMENT

L'unité de traitement du processeur est principalement composée d'une Unité Arithmétique et Logique, d'un banc de registres et d'une mémoire de données, ainsi que de composants annexes (multiplexeurs, extension de signe...)

### Unité Arithmétique et Logique (UAL ou ALU)

L'Unité Arithmétique et Logique (UAL) 32 bits possède :

- OP : Signal de commande sur 2 bits
- A, B : Deux bus 32 bits en entrée
- S : Bus 32 bits en sortie
- N : drapeau de sortie sur 1 bit signifiant que le résultat de la dernière opération est négatif
- Z : drapeau de sortie sur 1 bit signifiant que le résultat de la dernière opération est égale à zéro

Les valeurs du signal de commande OP déterminent les opérations implémentées dans l'UAL :

- ADD :  $Y = A + B$ ; OP = "00";
- B :  $Y = B$ ; OP = "01";
- SUB :  $Y = A - B$ ; OP = "10";
- A :  $Y = A$ ; OP = "11";

A l'issue de chaque opération :

- le drapeau N prend la valeur 1 si le résultat est strictement Négatif, et 0 sinon.
- le drapeau Z prend la valeur 1 si le résultat est strictement égal à zéro, et 0 sinon.

*Conseil : La librairie `ieee.numeric_std.all` permet d'utiliser les opérateurs '+' et '-'*

### Ban de registre

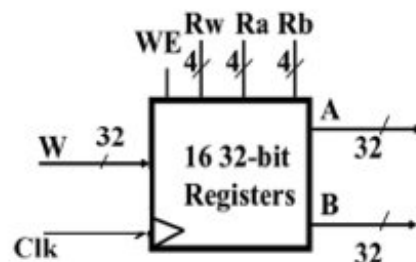
Le banc possède 16 registres de 32 bits. Il comporte les entrées/ sorties suivantes :

- CLK: Horloge,
- **Reset** : reset asynchrone (actif à l'état haut) non représenté sur le schéma
- W: Bus de données en écriture sur 32 bits
- RA: Bus d'adresses en lecture du port A sur 4 bits
- RB: Bus d'adresses en lecture du port B sur 4 bits
- RW: Bus d'adresses en écriture sur 4 bits
- WE: Write Enable sur 1 bit
- A: Bus de données en lecture du port A
- B: Bus de données en lecture du port B

La **lecture** des registres se fait de manière **asynchrone et simultanée** :

- Le bus de sortie A prend la valeur du registre N° RA
- Le bus de sortie B prend la valeur du registre N° RB

L'**écriture** des registres se fait de manière **synchrone**, sur le front montant de l'horloge. Elle est commandée par le signal WE :



- Si WE = 1, au front montant on recopie la valeur du bus W dans le registre n° RW
- Si WE = 0, au front montant il n'y a pas d'écriture effectuée

## 1.1 Travail à réaliser

**1.1.1 Décrire et simuler ces modules en VHDL comportemental à l'aide d'un banc de test et un script de simulation. Reporter les résultats de simulations sur votre compte-rendu.**

Le banc de registres sera déclaré comme étant un tableau de std\_logic\_vector. Son initialisation sera réalisée à l'aide d'une fonction (voir code ci-dessous)

```

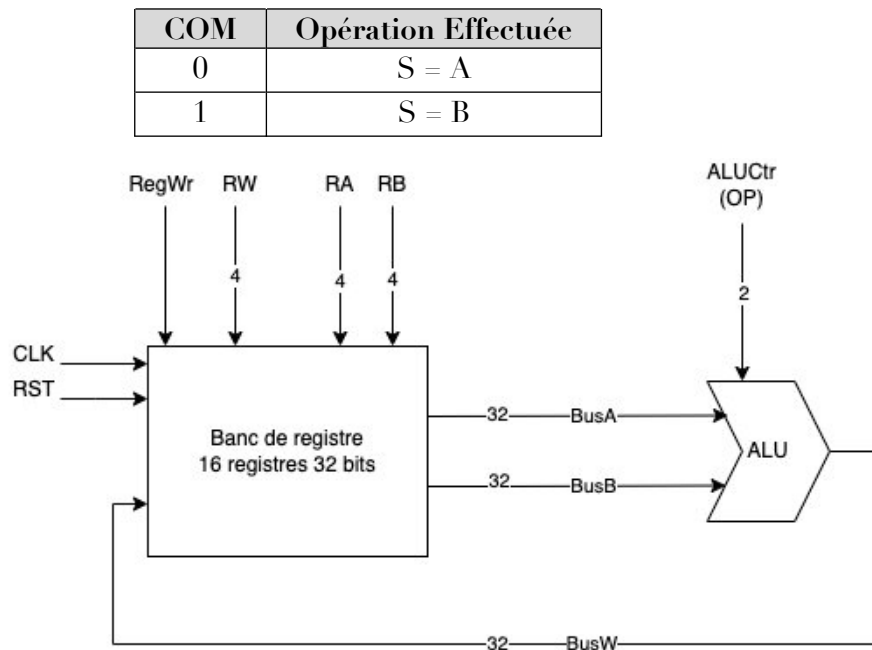
-- Declaration Type Tableau Memoire
type table is array(15 downto 0) of std_logic_vector(31 downto 0);

-- Fonction d'Initialisation du Banc de Registres
function init_banc return table is
variable result : table;
begin
  for i in 14 downto 0 loop
    result(i) := (others=>'0');
  end loop;
  result(15) := X"00000030";
  return result;
end init_banc;

-- Déclaration et Initialisation du Banc de Registres 16x32 bits
signal Banc: table:=init_banc;
  
```

*Conseil : Il faut impérativement éviter que la variable d'indexation du tableau soit un entier négatif, non initialisé, ou supérieur à la taille du tableau. Si un de ces cas se produit, la simulation s'arrêtera sur une erreur fatale.*

**1.1.2 Assembler l'UAL et le banc comme sur le schéma ci-dessous pour valider l'unité de traitement.**



**1.1.3 Écrire un banc de test et un script de simulation pour valider par simulation le bon fonctionnement des opérations suivantes :**

- $R(1) = R(15)$
- $R(1) = R(1) + R(15)$
- $R(2) = R(1) + R(15)$
- $R(3) = R(1) - R(15)$
- $R(5) = R(7) - R(15)$

**Reporter les résultats de simulations sur votre compte-rendu.** Pour la vérification de ces opérations, on vous conseille d'utiliser les alias et des assertions qui sont toujours plus efficaces qu'une vérification visuelle des chronogrammes.

## Multiplexeur 2 vers 1

Ce multiplexeur dispose d'un paramètre générique N fixant la taille des données en entrée et en sortie

- A, B : Deux bus sur N bits en entrée
- COM: Commande de sélection sur 1 bit
- S : Bus sur N bits en sortie

Les valeurs du signal COM fixe le comportement de la sortie

## Extension de signe

Ce module permet d'étendre en sortie sur 32 bits le signe d'une entrée codée sur N bits. Il possède donc un paramètre générique fixant la valeur de N. Il comporte également

- E : Bus sur N bits en entrée
- S : Bus sur 32 bits en sortie

## Mémoire de données

Cette mémoire permet de charger et stocker 64 mots de 32 bits. Il comporte les entrées sorties suivantes :

**CLK** : Horloge

**Reset** : reset asynchrone (actif à l'état haut) non représentés sur le schéma.

**DataIn** : Bus de données en écriture sur 32 bits

**DataOut** : Bus de données en lecture sur 32 bits

**Addr** : Bus d'adresses en lecture et écriture sur 6 bits

**WrEn** : Write Enable sur 1 bit

La **lecture** des registres se fait de manière **asynchrone et simultanée** :

Le bus de sortie **DataOut** porte la valeur de la données N° **Addr**

L'**écriture** des registres se fait de manière **synchrone**, sur le front montant de l'horloge. Elle est commandée par le signal WE :

- Si WE = 1, au front montant on recopie la valeur du bus **DataIn** dans le registre n° Addr
- Si WE = 0, au front montant il n'y a pas d'écriture effectuée

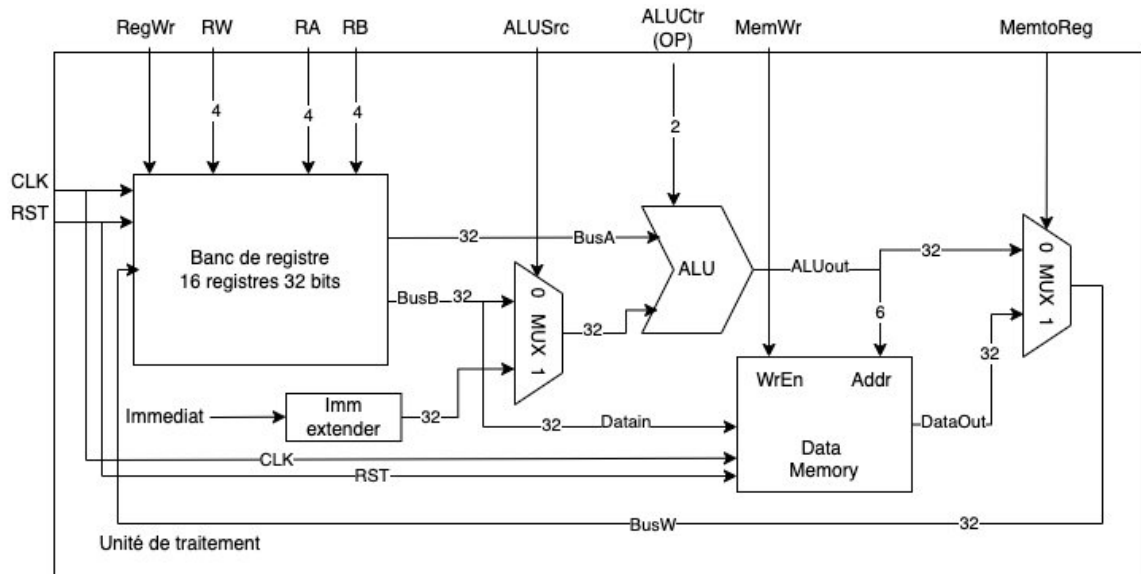
*Conseil: Ce module est très proche du banc de registre et peut être conçu à partir de ce dernier.*

## 1.2 Travail à réaliser

**1.2.1 Décrire et simuler ces modules en VHDL comportemental. Reporter les résultats de simulations sur votre compte-rendu.**

## 1.3 Assemblage Unité de Traitement

Les modules décrits précédemment doivent être assemblés conformément au schéma ci-dessous :



Le signal provenant de l'Alu étant sur 32 bits, la réduction avec les 6 bits de Addr peut être réalisée au niveau du **Port Map** (Addr => SignalAluOut (5 downto 0), ...)

**1.3.1 Écrire un module VHDL effectuant l'assemblage de l'unité de traitement.**

**1.3.2 Écrire un banc de test et un script de simulation pour valider le bon fonctionnement de :**

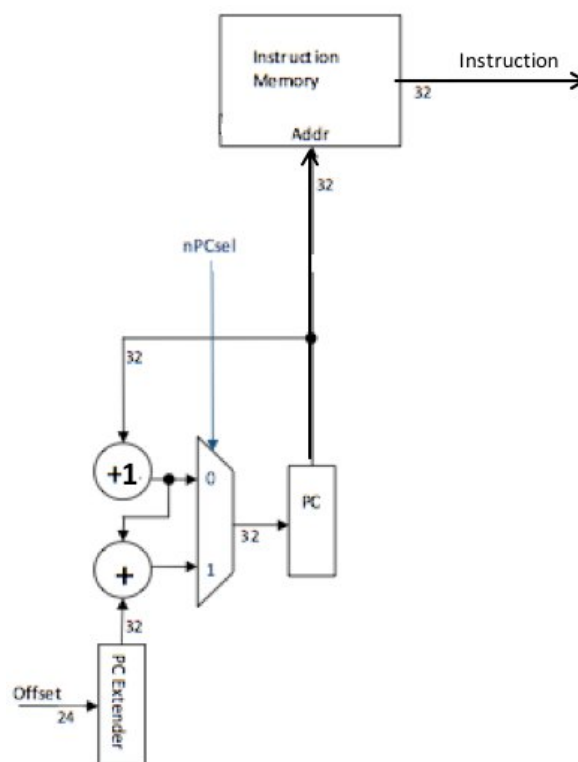
- L'addition de 2 registres
- L'addition d'1 registre avec une valeur immédiate
- La soustraction de 2 registres
- La soustraction d'une valeur immédiate à 1 registre
- La copie de la valeur d'un registre dans un autre registre
- L'écriture d'un registre dans un mot de la mémoire.
- La lecture d'un mot de la mémoire dans un registre.

**Reporter les résultats de simulations sur votre compte-rendu.**

## PARTIE 2 – UNITE DE GESTION DES INSTRUCTIONS

L'unité de gestion des instructions 32 bits, représentée sur la figure ci-dessous, possède :

- Une mémoire d'instruction de 64 mots de 32 bits similaire à celle de l'unité de traitement, celle-ci est fourni en annexe dans le fichier **instruction\_memory.vhd**
- Il n'y a pas de bus de données en écriture ni de Write Enable.
- Un registre 32 bits (Registre PC). Ce registre possède en entrée une horloge et un reset asynchrone (actif à l'état haut) non représentés sur le schéma.
- Une unité d'extension de 24 à 32 bits signés similaire au module décrit précédemment
- Une unité de mise à jour du compteur de programme PC suivant le signal de contrôle nPCsel :
  - Si nPCsel = 0 alors  $PC = PC + 1$
  - Si nPCsel = 1 alors  $PC = PC + 1 + \text{SignExt}(\text{offset})$



Vous pouvez décrire ce composant de manière complètement structurelle ou alors choisir d'utiliser une description comportementale pour la partie de génération des nouvelles adresses.

**2.1 Décrire et simuler l'unité de gestion des instructions avec un banc de test et un script de simulation. Reporter les résultats de simulations sur votre compte-rendu.**

## PARTIE 3 – UNITE DE CONTROLE

L'unité de contrôle est constituée d'un registre 32 bits et d'un décodeur combinatoire.

### Registre 32 bits avec commande de chargement

Ce registre servira à stocker l'état du processeur (Processor State Register ou PSR) et doit permettre de stocker sur RegAff une valeur à afficher sur les LEDs ou les afficheurs 7 segments lors des instructions STR.

Il possède les entrées/sorties suivantes :

- DATAIN : Bus de chargement sur 32 bits
- RST : Reset asynchrone, actif à l'état haut
- CLK : Horloge
- WE : Commande de chargement
- DATAOUT : Bus de sortie sur 32 bits

Si WE=1, le registre mémorise la valeur placée sur le bus DATAIN.

Si WE=0, le registre conserve sa valeur précédente (état mémoire)

### Décodeur d'instructions

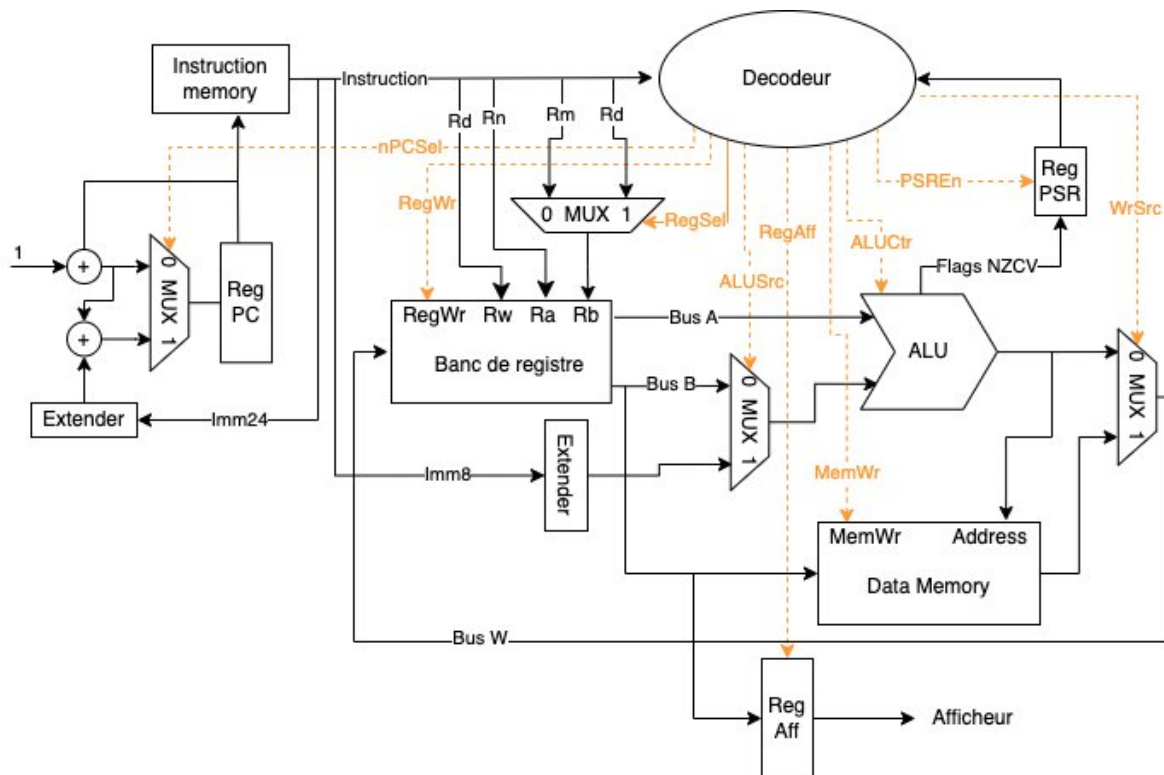
Ce module combinatoire génère les signaux de contrôle de l'unité de traitement, de l'unité de gestion des instructions, ainsi que des registre PSR, tous décrits précédemment.

Les valeurs de ces commandes dépendent de l'instruction récupérée dans la mémoire instructions, et éventuellement de l'état du registre PSR. La structure binaire des différentes instructions est décrite en Annexe.

Les commandes à générer concernent :

- Le multiplexeur d'entrée du registre PC (**nPC\_SEL**)
- La commande de chargement du registre PSR (**PSREn**)
- Les bus d'adresse **Rn, Rm, Rd**, ainsi que le **WE** du banc de registres (**RegWr**)
- La commande **RegSel** du multiplexeur situé en amont du bus d'adresse RB de ce banc.
- La commande OP de l'UAL (**ALUCtrl**)
- La commande du multiplexeur (**MemToReg**) qui permet de choisir le signal à écrire dans le banc de registre.
- La commande **ALUSrc** du multiplexeur situé en amont de l'entrée B de l'UAL.
- La commande **WrSrc** du multiplexeur situé en sortie de l'UAL et de la mémoire de données.
- Le Write Enable de la mémoire de données (**MemWr**)
- Le signal qui permet d'afficher une valeur en sortie lors de l'opération STR (**RegAff**)

Schéma global du processeur :



Pour la conception de ce décodeur, on s'appuiera sur le schéma d'ensemble du processeur présenté ci-dessus, du tableau de commande (voir page suivante) ainsi que sur le descriptif du jeu d'instructions fournis en annexe.

Pour décrire ce décodeur d'instructions, on déclarera tout d'abord le type énuméré suivant :

```
type enum_instruction is (MOV, ADDi, ADDr, CMP, LDR, STR, BAL, BLT);
signal instr_courante: enum_instruction;
```

On pourra ensuite utiliser deux process dans l'architecture

- Un process sensible sur la sortie de la mémoire instructions (nommée instruction sur le schéma en annexe) dont le rôle est de fixer la valeur du signal instr\_courante
- Un process sensible sur les signaux **instructions** et **instr\_courante** qui donnera la valeur des commandes des registres et opérateurs du processeur.

**3.1 Compléter le tableau « valeurs des commandes » se trouvant ci-dessous, qui synthétisera les actions du décodeur en fonction des instructions.**

**3.2 Décrire en VHDL les deux modules de l'unité de contrôle : Le décodeur d'instructions et le registre PSR sur 32 bits, s'il reçoit une commande de chargement, fera l'acquisition des drapeaux N et Z de l'ALU sur les bits de poids fort.**



## Valeurs des commandes

| INSTRUCTION | nPCSel | RegWr | ALUSrc | ALUCtr | PSREn | MemWr | WrSrc | RegSel | RegAff |
|-------------|--------|-------|--------|--------|-------|-------|-------|--------|--------|
| ADDi        |        |       |        |        |       |       |       |        |        |
| ADDr        |        |       |        |        |       |       |       |        |        |
| BAL         |        |       |        |        |       |       |       |        |        |
| BLT         |        |       |        |        |       |       |       |        |        |
| CMP         |        |       |        |        |       |       |       |        |        |
| LDR         |        |       |        |        |       |       |       |        |        |
| MOV         |        |       |        |        |       |       |       |        |        |
| STR         |        |       |        |        |       |       |       |        |        |

## PARTIE 4 – ASSEMBLAGE ET VALIDATION DU PROCESSEUR

Il faut à présent finaliser la modélisation du processeur en assemblant ses trois unités principales.

- L'unité de gestion des instructions
- L'unité de traitement
- L'unité de contrôle

4.1 Compléter l'unité de traitement conçue précédemment en ajoutant un multiplexeur à 2 entrées sur 4 bits piloté par le signal de commande RegSel généré par l'unité de contrôle. Ce multiplexeur sera placé en entrée de l'adresse RB du banc de registres, comme cela est représenté sur le schéma du processeur.

4.2 Assembler le processeur à partir de ses trois unités.

4.3 Simuler l'exécution du programme de test par le processeur et vérifier son bon fonctionnement. Il sera nécessaire d'initialiser quelques données dans la Mémoire Data (de 0x10 à 0x1A)

### Programme de test

```
0x0 main : MOV R1,#0x10      ; --R1 <= 0x10
0x1      : MOV R2,#0         ; --R2 <= 0
0x2 loop : LDR R0,0(R1)      ; --R0 <= DATA_MEM[R1]
0x3      : ADD R2,R2,R0      ; --R2 <= R2 + R0
0x4      : ADD R1,R1,#1      ; --R1 <= R1 + 1
0x5      : CMP R1,0x1A       ; -- R1 - 0x1A, mise à jour de N
0x6      : BLT loop          ; --branchement à _loop si R1 inferieur a 0x1A
0x7 end   : STR R2,0(R1)     ; --DATA_MEM[R1] <= R2
0x8      : BAL main          ; --branchement à _main
```

## Code assembleur de la partie 4 : instruction\_memory.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity instruction_memory is
  port(
    PC: in std_logic_vector (31 downto 0);
    Instruction: out std_logic_vector (31 downto 0)
  );
end entity;

architecture RTL of instruction_memory is
  type RAM64x32 is array (0 to 63) of std_logic_vector (31 downto 0);

  function init_mem return RAM64x32 is
    variable result : RAM64x32;
  begin
    for i in 63 downto 0 loop
      result (i):=(others=>'0');
    end loop;
    -- PC          -- INSTRUCTION -- COMMENTAIRE
    result (0):=x"E3A01010";-- 0x0 _main -- MOV R1,#0x10 -- R1 = 0x10
    result (1):=x"E3A02000";-- 0x1      -- MOV R2,#0x00 -- R2 = 0
    result (2):=x"E4110000";-- 0x2 _loop -- LDR R0,0(R1) -- R0 = DATAMEM[R1]
    result (3):=x"E0822000";-- 0x3      -- ADD R2,R2,R0 -- R2 = R2 + R0
    result (4):=x"E2811001";-- 0x4      -- ADD R1,R1,#1 -- R1 = R1 + 1
    result (5):=x"E351001A";-- 0x5      -- CMP R1,0x1A -- Flag = R1-0x1A, si R1 <= 0x1A
    result (6):=x"BAFFFFFFB";-- 0x6      -- BLT loop -- PC =PC+1+(-5) si N = 1
    result (7):=x"E4012000";-- 0x7      -- STR R2,0(R1) -- DATAMEM[R1] = R2
    result (8):=x"EAFFFFF7";-- 0x8      -- BAL main -- PC=PC+1+(-9)
    return result;
  end init_mem;

  signal mem: RAM64x32 := init_mem;

begin
  Instruction <= mem(to_integer(unsigned (PC)));
end architecture;
```

## PARTIE 5 : TEST DU PROCESSEUR SUR CARTE FPGA

Dans cette partie, vous allez implémenter votre processeur monocycle sur une carte FPGA.

Pour vérifier que l'exécution du programme de test se fait sans erreur, vous allez afficher sur les afficheurs 7 segments la valeur qui se trouve en sortie du registre **Reg Aff**.

Pour cela, il faut rajouter 4 décodeurs 7 segments pour décoder les bits 0 à 15 de la sortie du registre **Reg Aff** afin d'afficher cette valeur en hexadécimal sur 4 digits.

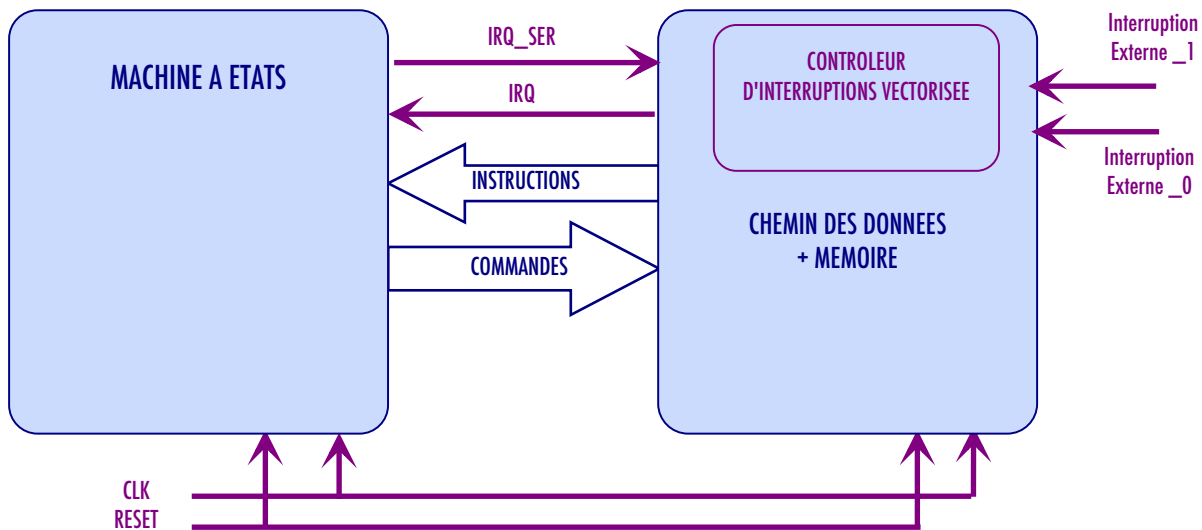
### Travail à réaliser :

5.1 Créer un projet sur Quartus, instancier le processeur et les 4 décodeurs 7 segments dans un fichier **top\_level.vhd**. Faire l'assignement des pins, lancer la synthèse et programmer la carte

5.2 Vérifier que vous trouvez la valeur attendue sur les afficheurs 7 segments.

## Partie 6 : Processeur Multicycle

Nous allons à présent faire évoluer l'architecture du processeur mono-cycle modélisé précédemment afin que les instructions soient exécutées sur plusieurs cycles. Le synopsis de son architecture est donné ci-dessous :



- Le bloc « Chemin de Données + Mémoire » est inspiré de la version mono-cycle du processeur avec quelques ajustements. Il inclut un contrôleur d'interruptions qui permet de gérer le traitement de deux interruptions externes (qui seront connectées à deux boutons poussoirs de la carte FPGA).

- Le bloc « Machine à Etats » réalise le décodage des instructions et permet leur exécution en configurant les différents registres et opérateurs du chemin de données.

### Le but de de cette partie :

- Reprendre le chemin de données du processeur mono-cycle pour l'adapter au multi-cycles.
- Décrire en VHDL la machine à états du processeur et de l'interfacer au chemin de données dont le code vous sera donné.
- Implémenter le processeur complet sur la carte FPGA

# 1/ – PROJET SUR QUARTUS

## A FAIRE

- Copier les sources du processeur multi-cycles depuis Moodle.
- Créer un projet sur Quartus pour la carte FPGA. Pour le moment, rajoutez les 3 fichiers source VHDL (Menu Project → Add/Remove Files in Project):
  - a. `top_level` : le fichier principal qui instancie le processeur et le connecte aux entrées/sorties de la carte FPGA
  - b. `ARM` : le modèle du processeur. On y instancie le chemin de données et la MAE.
  - c. `DATAPATH` : le chemin de données du processeur multi-cycles.
- 2) Ajouter aux sources du projet (Menu Project → Add/Remove Files in Project) les modules de base développés pour le processeur mono-cycle, à savoir :
  - a. UAL (ou ALU)
  - b. Banc de registres
  - c. Mux 2→1
  - d. Extension de signe
  - e. Registre 32 bits avec commande de chargement

## Modules de base additionnels pour le chemin de données :

### 1.1/ Multiplexeur 4 → 1

#### Entrées/Sorties :

- A, B, C, D: Entrées sur 32 bits
- COM: Commande sur 2 bits.
- S: Sortie sur 32 bits.

#### Fonctionnement:

Ce multiplexeur 4→1 laisse passer en sortie l'entrée selon la valeur de la commande COM

| COM | Sortie du Multiplexeur |
|-----|------------------------|
| 00  | Entrée A               |
| 01  | Entrée B               |
| 10  | Entrée C               |
| 11  | Entrée D               |

### 1.2/ Registre 32

#### Entrées/Sorties :

- DATAIN: Entrée sur 32 bits
- CLK: Horloge
- RST: Reset asynchrone, actif à l'état haut
- DATAOUT: Sortie sur 32 bits.

### 1.3/ Memoire 64 x 32 bits

La mémoire du processeur DualPortRAM.vhd est fourni dans le dossier **src**. Il s'agit d'une mémoire double port initialisé au reset avec le code assembleur à exécuter.

### A FAIRE

- 1) Créer de nouvelles sources VHDL pour décrire les modules additionnels (Mux 4→1, Registre 32 bits).
- 2) Ajouter ces sources au projet.





## VIC : Controleur d'interruption vectorisé

### Entrées :

- CLK: Horloge
- RESET : Reset asynchrone, actif à l'état haut
- IRQ\_SERV : Acquittement de l'interruption venant de la MAE
- IRQ0, IRQ1 : Requêtes d'interruption

### Sorties :

- IRQ: Requête d'interruption envoyée à la MAE
- VICPC : Adresse de début du sous-programme d'interruption (sur 32 bits)

### Fonctionnement :

- Ce module échantillonne les valeurs de IRQ0 et IRQ1 afin de connaître la dernière (IRQi(n)) et l'avant dernière (IRQi(n-1)) valeur de ces deux signaux.
- Si on détecte une transition montante (IRQi(n)=1 ET IRQi(n-1)=0) sur l'un de ces signaux, on force à l'état haut un signal IRQ0\_memo ou IRQ1\_memo, signalant une requête d'interruption.
- IRQ1\_memo et IRQ0\_memo restent à l'état haut tant que l'on n'a pas reçu d'acquittement sur l'entrée IRQ\_SERV.
- S'il n'y a aucune requête d'interruption, VICPC est forcé à 0.
  - Si IRQ0\_memo = 1, on force la sortie VICPC à la valeur 0x9 (adresse de début de ce sous-programme d'interruption)
  - Si IRQ1\_memo = 1, on force la sortie VICPC à la valeur 0x15 (adresse de début de ce sous-programme d'interruption)
  - L'IRQ0 est prioritaire sur l'IRQ1.
  - La sortie IRQ envoyée à la MAE est un OU logique entre IRQ1\_memo et IRQ0\_memo.

**Signal envoyé à la MAE :**                    **IRQ**  
**Commande à générer par la MAE:**       **IRQ\_SERV.**

## A FAIRE

- Créer une nouvelle source VHDL pour décrire le VIC.
- Simuler ce composant à l'aide d'un banc de test et un script de simulation.
- Ajouter cette source au projet.

### MUX\_PC: Multiplexeur d'entrée du registre PC

Ce bloc est un multiplexeur 4→1. Les entrées et la sortie sont sur 32 bits.

Entrées :

- A: Sortie de l'ALU
- B: Sortie du registre ALUOUT
- C: Sortie du registre LR
- D: Sortie du VIC (Adresse du sous-programme d'interruption)
- PCSel: Commande sur 2 bits.

Sorties :

- S: Connectée à l'entrée du registre PC

Fonctionnement:

Ce multiplexeur 4→1 permet de sélectionner la donnée à charger dans le registre PC. Il est commandé par un signal PCSel sur 2 bits

| PCSel | Sortie du Multiplexeur    |
|-------|---------------------------|
| 00    | Sortie de l'ALU           |
| 01    | Sortie du registre ALUOUT |
| 10    | Sortie du registre LR     |
| 11    | Sortie du VIC             |

**Commande à générer par la MAE: PCSel (2 bits)**

### PC: Registre PC

Fonctionnement

Ce registre 32 bits avec commande de chargement contient l'adresse de la prochaine instruction. Il charge une donnée en entrée si la commande PCWrEn est mise à 1, et conserve sa valeur précédente si la commande est à 0.

**Commande à générer par la MAE: PCWrEn**

### LR: Link Register

Fonctionnement :

Ce registre 32 bits avec commande de chargement sauvegarde la valeur du registre PC lors du traitement d'une interruption. Le chargement de l'adresse de retour au programme principal est commandé par le signal LRWrEn, actif à 1. Le registre conserve sa valeur sinon.

**Commande à générer par la MAE: LRWrEn**

### Mux\_MEM : Multiplexeur du Bus d'adresses de la mémoire

#### Fonctionnement :

Ce multiplexeur 2→1 sur 6 bits permet de sélectionner la valeur à positionner sur le bus d'adresses de la mémoire. Il est commandé par le signal **AdrSel**.

| AdrSel | Sortie du Multiplexeur                 |
|--------|----------------------------------------|
| 0      | Sortie du registre PC (5 downto 0)     |
| 1      | Sortie du registre ALUOUT (5 downto 0) |

**Commande à générer par la MAE:   AdrSel**

### MEMORY: Memoire interne du processeur

#### Fonctionnement :

La mémoire contient le programme à exécuter ainsi que toutes les données nécessaires. D'une capacité de 64x32 bits, elle possède un port en lecture et un port en écriture. Le bus d'adresses est commun à ces deux ports.

La lecture est commandée par le signal **MemRdEn** actif à l'état haut.

L'écriture est commandée par le signal **MemWrEn**, actif à l'état haut

**Commande à générer par la MAE:   MemRdEn, MemWrEn**

### IR : Registre Instruction

#### Fonctionnement :

Ce registre 32 bits avec commande de chargement contient l'instruction courante du programme. Il charge une donnée provenant de la mémoire si la commande **IRWrEn** est à l'état haut, et conserve sa valeur sinon.

**Commande à générer par la MAE:   IRWrEn**

### DR : Registre de Données Mémoire

#### Fonctionnement :

Ce registre 32 bits stocke une donnée issue de la mémoire. Le chargement se fait à chaque cycle d'horloge.

**Commande à générer par la MAE:   Aucune**

### MUX\_REG\_RB : Multiplexeur du Bus d'adresses RB du banc de registres

#### Fonctionnement :

Ce multiplexeur 2→1 sur 4 bits permet de sélectionner la valeur à positionner sur le bus d'adresses RB du banc de registres. Sa commande est générée par l'instruction stockée dans le registre IR selon l'équation suivante :  $COM = NOT [IR(27) OR IR(20)] AND IR(26)$ .

| COM | Sortie du Multiplexeur               |
|-----|--------------------------------------|
| 0   | Sortie du registre IR (3 downto 0)   |
| 1   | Sortie du registre IR (15 downto 12) |

**Commande à générer par la MAE:   Aucune**

### MUX\_REG\_BUSW : Multiplexeur du Bus des données W du banc des registres

#### Fonctionnement :

Ce multiplexeur 2→1 sur 32 bits permet de sélectionner la valeur à positionner sur le bus de données W du port d'écriture du banc de registres. Il est commandé par le signal WSel.

| WSel | Sortie du Multiplexeur    |
|------|---------------------------|
| 0    | Sortie du registre DR     |
| 1    | Sortie du registre ALUOUT |

**Commande à générer par la MAE:   WSel**

### Register File : Banc de registres

#### Fonctionnement :

Le banc de registres contient 16 registres de 32 bits, accessibles via deux ports de lecture et un port d'écriture. L'écriture est commandée par le signal RegWrEn.

**Commande à générer par la MAE :   RegWrEn**

### Ext\_8 : Extension de données 8 → 32

#### Fonctionnement :

Extension signée des 8 bits de poids faible du registre IR sur 32 bits

**Commande à générer par la MAE :   Aucune**

**Ext 24 : Extension de données 24 → 32****Mux\_ALU\_A : Multiplexeur entrée A de l'ALU**

Fonctionnement :

Ce multiplexeur 2→1 sur 32 bits permet de sélectionner la valeur à positionner sur l'entrée A de l'ALU. Il est commandé par le signal ALUSelA.

| ALUSelA | Sortie du Multiplexeur |
|---------|------------------------|
| 0       | Sortie du registre PC  |
| 1       | Sortie du registre A   |

**Commande à générer par la MAE:    ALUSelA**

**A : Registre de sortie du port A du banc de registres**

Fonctionnement :

Ce registre 32 bits stocke à chaque cycle la valeur du bus de données du port de lecture A du banc de registres.

**Commande à générer par la MAE :    Aucune**

**B : Registre de sortie du port B du banc de registres**

Fonctionnement :

Ce registre 32 bits stocke à chaque cycle la valeur du bus de données du port de lecture B du banc de registres.

**Commande à générer par la MAE :    Aucune**

**ALU\_Out : Registre de sortie de l'ALU**

Fonctionnement :

Ce registre 32 bits stocke à chaque cycle d'horloge la sortie de l'ALU.

**Commande à générer par la MAE:    Aucune**

### Mux\_ALU\_B : Multiplexeur entrée B de l'ALU

Fonctionnement :

Ce multiplexeur 4→1 sur 32 bits permet de sélectionner la valeur à positionner sur l'entrée B de l'ALU. Il est commandé par le signal ALUSelB.

| ALUSelB | Sortie du Multiplexeur |
|---------|------------------------|
| 00      | Sortie du registre B   |
| 01      | Sortie du module EXT8  |
| 10      | Sortie du module EXT24 |
| 11      | 1 (valeur constante)   |

Commande à générer par la MAE: **ALUSelB (2 bits)**

### ALU : Unité Arithmétique et Logique

Fonctionnement :

L'ALU permet d'effectuer une opération, en fonction de la commande ALUOP sur 2 bits. En plus du résultat, il fournit des drapeaux indicateurs sur la nature du résultat (en particulier le flag N).

| ALUOP | Opération |
|-------|-----------|
| 00    | $A + B$   |
| 01    | B         |
| 10    | $A - B$   |
| 11    | A         |

Commande à générer par la MAE: **ALUOP (2 bits)**

### Mux\_CPSR : Multiplexeur d'entrée du Registre CPSR

Fonctionnement :

Ce multiplexeur 2→1 sur 32 bits permet de sélectionner la valeur à positionner en entrée du registre CPSR. Il est commandé par le signal CPSRSel.

| CPSRSel | Sortie du Multiplexeur                                                                   |
|---------|------------------------------------------------------------------------------------------|
| 0       | 32 bits comprenant le drapeau N de l'ALU (MSB) et les bits 30 à 0 du registre CPSR (LSB) |
| 1       | Sortie du registre SPSR                                                                  |

**Commande à générer par la MAE: CPSRSel**

### CPSR : Current Processor Status Register

Fonctionnement :

Ce registre 32 bits avec commande de chargement stocke l'état courant du processeur. Le chargement est commandé par le signal CPSRWrEn, actif à l'état haut.

**Commande à générer par la MAE: CPSRWrEn**

### SPSR : Saved Processor Status Register

Fonctionnement :

Ce registre 32 bits à commande de chargement sauvegarde l'état du processeur en cas de traitement d'une interruption. Le chargement est commandé par le signal SPSRWrEn, actif à l'état haut.

**Commande à générer par la MAE: SPSRWrEn**

## A FAIRE

- 1) Compléter le fichier VHDL DATAPATH pour y instancier tous les blocs de base du chemin de données, conformément au cahier des charges ci-dessus.

## 3 : CODAGE VHDL DE LA MACHINE A ETATS

Le rôle de la MAE est donc d'effectuer le décodage des instructions envoyées par la mémoire du processeur et d'en mettre en œuvre l'exécution. On trouvera ci-dessous une vue externe de ce composant.

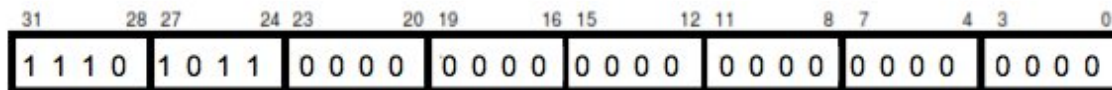
### Descriptif des signaux d'E/S :

- CLK : Horloge
- RST : Reset Asynchrone
- IRQ : Requête d'interruption provenant du VIC
- IRQ\_SERV : Drapeau d'interruption

- INST\_MEM: Instruction (32 bits) à décoder. Vient de la sortie de la mémoire
- INST\_REG : Instruction (32 bits) à décoder. Vient de la sortie du registre IR (\*)
- N: Drapeau N de l'ALU (pour les instructions de branchement conditionnel)
- COMMANDES : Commandes des registres et des opérateurs du chemin de données (cf. plus haut)

(\*) En raison de l'exécution sur plusieurs cycles des instructions, il est nécessaire d'avoir accès à l'instruction depuis ces deux points du chemin de données.

Le sous-ensemble du jeu d'instruction est le même que celui du processeur mono-cycle. On y ajoute simplement l'instruction BX qui permet la restauration du contexte à la fin du traitement d'une interruption. Son code binaire est le suivant :



## Fonctionnement

Le fonctionnement général de la MAE est donné par les graphes d'état des annexes. Ce graphe montre comment implémenter la gestion des interruptions en rajoutant trois états et des conditions supplémentaires sur IRQ et un signal interne ISR dont le comportement est le suivant :

- ISR est un signal synchrone sur l'horloge
- ISR passe à 1 lorsque la MAE provoque une sauvegarde de contexte.
- ISR passe à 0 lorsque la MAE provoque une restauration de contexte.

Chaque état implique un positionnement bien spécifique des signaux de commande à envoyer au chemin de données.

## Programme de Test

Le programme de test est le suivant. Le programme principal fait la somme de 10 cases mémoires qui contiennent des valeurs allant de 1 à 10. Le résultat attendu est donc 55 en décimal, 0x37 en hexa.



Les sous-programmes d'interruption incrémentent la valeur de l'une des cases mémoire, modifiant ainsi le total en sortie.

#### Programme principal:

```

0x0  _main :      MOV R1,#0x20    ; --R1 <= 0x20
0x1      MOV R2,#0              ; --R2 <= 0
0x2  _loop :      LDR R0,0(R1)    ; --R0 <= MEM[R1]
0x3      ADD R2,R2,R0          ; --R2 <= R2 + R0
0x4      ADD R1,R1,#1           ; --R1 <= R1 + 1
0x5      CMP R1,0x2A           ; --? R1 = 0x2A
0x6      BLT loop              ; --branchement à _loop si R1
                                ; inferieur a 0x2A
                                ;
      _end   :
0x7      STR R2,0(R1)           ; --MEM[R1] <= R2
0x8      BAL main              ; --branchement à _main

```

#### ISR interruption 0 :

--sauvegarde du contexte - R15 correspond au pointeur de pile

```

0x9      STR R1,0(R15)          ; --MEM[R15] <= R1
0xA      ADD R15,R15,1          ; --R15 <= R15 + 1
0xB      STR R3,0(R15)          ; --MEM[R15] <= R3

```

--traitement

```

0xC      MOV R3,0x20            ; --R3 <= 0x20
0xD      LDR R1,0(R3)           ; --R1 <= MEM[R3]
0xE      ADD R1,R1,1            ; --R1 <= R1 + 1
0xF      STR R1,0(R3)           ; --MEM[R3] <= R1

```

--chargement du context

```

0x10     LDR R3,0(R15)          ; --R3 <= MEM[R15]
0x11     ADD R15,R15,-1         ; --R15 <= R15 - 1
0x12     LDR R1,0(R15)          ; --R1 <= MEM[R15]
0x13     BX                      ; -- instruction de fin

```

d'interruption

#### ISR interruption 1 :

--sauvegarde du contexte - R15 correspond au pointeur de pile

```

0x15     STR R4,0(R15)          ; --MEM[R15] <= R4
0x16     ADD R15,R15,1          ; --R15 <= R15 + 1
0x17     STR R5,0(R15)          ; --MEM[R15] <= R5

```

--traitement

```

0x18     MOV R5,0x20            ; --R5 <= 0x20
0x19     LDR R4,0(R5)           ; --R4 <= MEM[R5]
0x1A     ADD R4,R4,2            ; --R4 <= R1 + 2
0x1B     STR R4,0(R5)           ; --MEM[R5] <= R4

```

--chargement du contexte

```

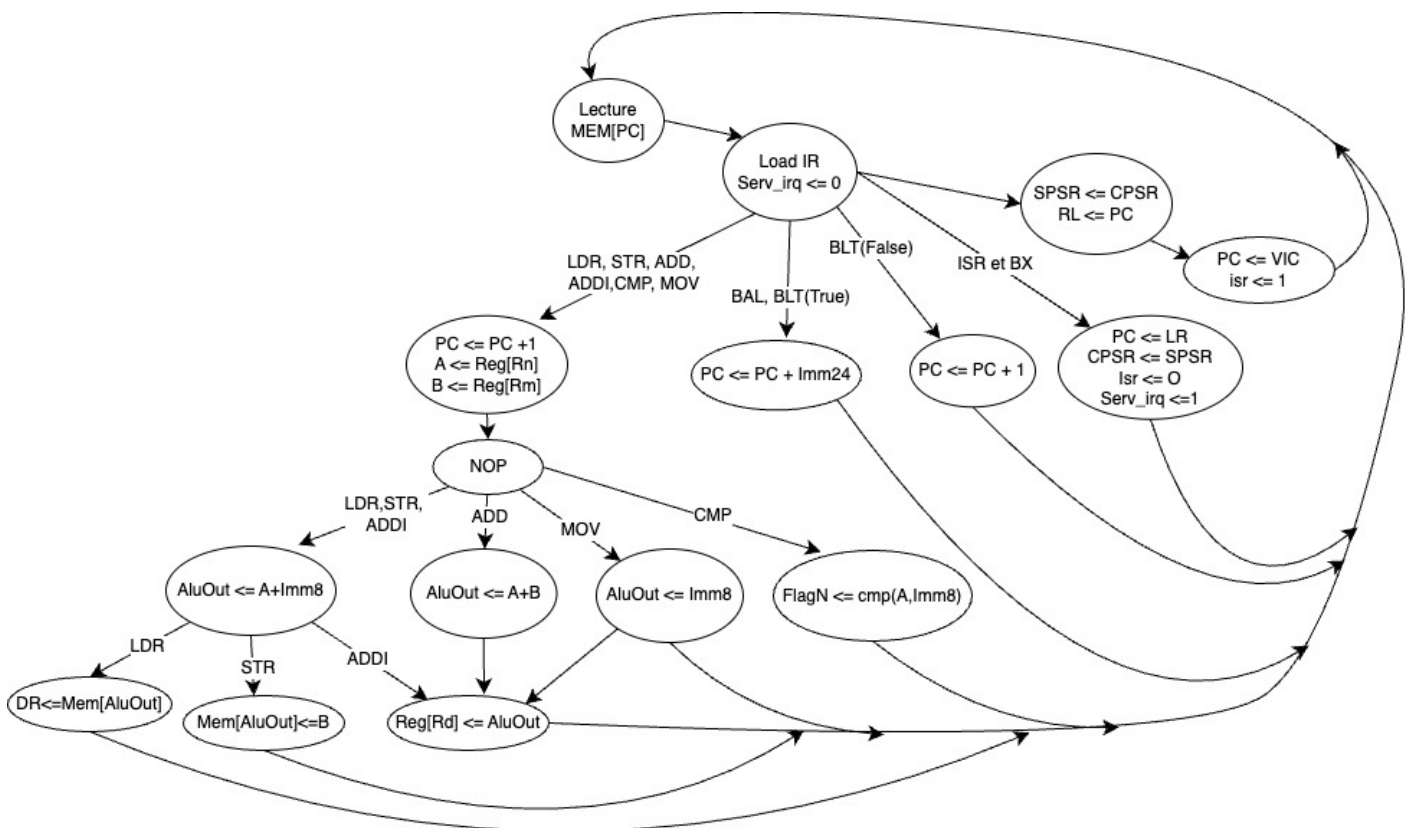
0x1C     LDR R5,0(R15)          ; --R5 <= MEM[R15]
0x1D     ADD R15,R15,-1         ; --R15 <= R15 - 1
0x1E     LDR R4,0(R15)          ; --R4 <= MEM[R15]
0x1F     BX                      ; -- instruction de fin

```

d'interruption

## Travail à Effectuer

- Compléter le fichier VHDL de la MAE du processeur (MAE.vhd), conformément au graphe donné ci-dessous. Ajouter cette source au projet. Instancier la MAE dans le fichier ARM.
- Tester le processeur ARM à l'aide d'un banc de test avec Modelsim Altera. Vérifier que la sortie résultat passe à la valeur 55 (en décimal) et que lorsque vous générez une interruption le résultat change (+1 pour IRQ0 et +2 pour IRQ1)
- Vérifier avec l'Assignment Editor (menu Assignments) que les entrées/sorties du projet sont bien mappées sur la carte. Au besoin, importer le fichier .qsf de la carte FPGA (il se trouve dans le dossier **fit**).
- Compiler le projet (Menu Processing → Start Compiler). Noter sur le rapport et au tableau de synthèse le taux d'occupation des ressources du FPGA.
- Etudier le code du fichier top\_level pour comprendre comment piloter le processeur à l'aide des boutons, switches, LEDs et afficheurs de la carte. Implémenter le processeur sur la carte FPGA et vérifier le bon fonctionnement du processeur.



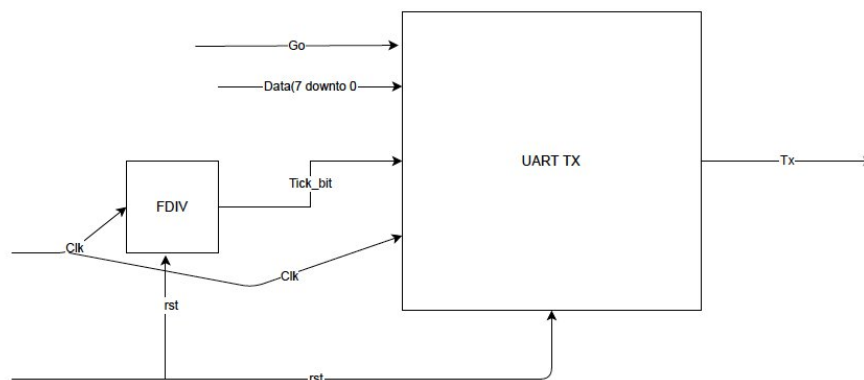
## Partie 7 : RAJOUTER UN PÉRIPHÉRIQUE UART

Dans cette partie, nous allons voir comment rajouter un périphérique UART à notre processeur. Vous allez commencer par décrire la partie opérative de l'UART et ensuite vous allez voir comment la connecter au processeur.

Pour tester cette UART, il faudra exécuter un petit code assembleur qui permet d'envoyer la phrase « HELLO WORLD ! » vers le terminal d'un PC.

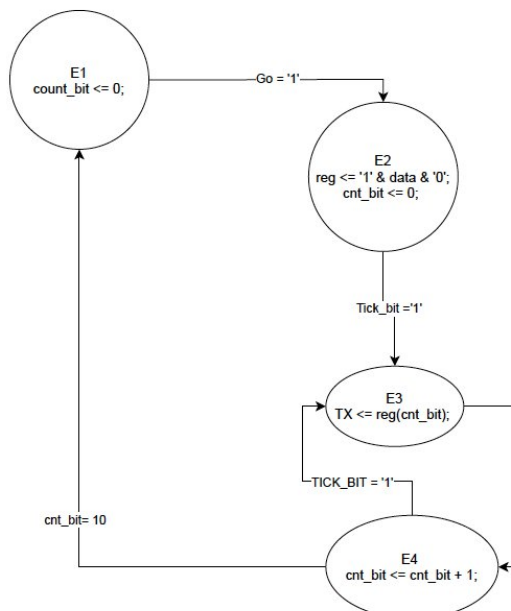
### Partie émission de l'UART :

Voici ci-joint le schéma et la MAE de la partie émission de l'UART :



signal reg : std\_logic\_vector(9 downto 0);  
signal cnt\_bit : integer range 0 to 15;

Reset :  
Tx <= '1';  
cnt\_bit <= 0;  
reg <= (others => '0');

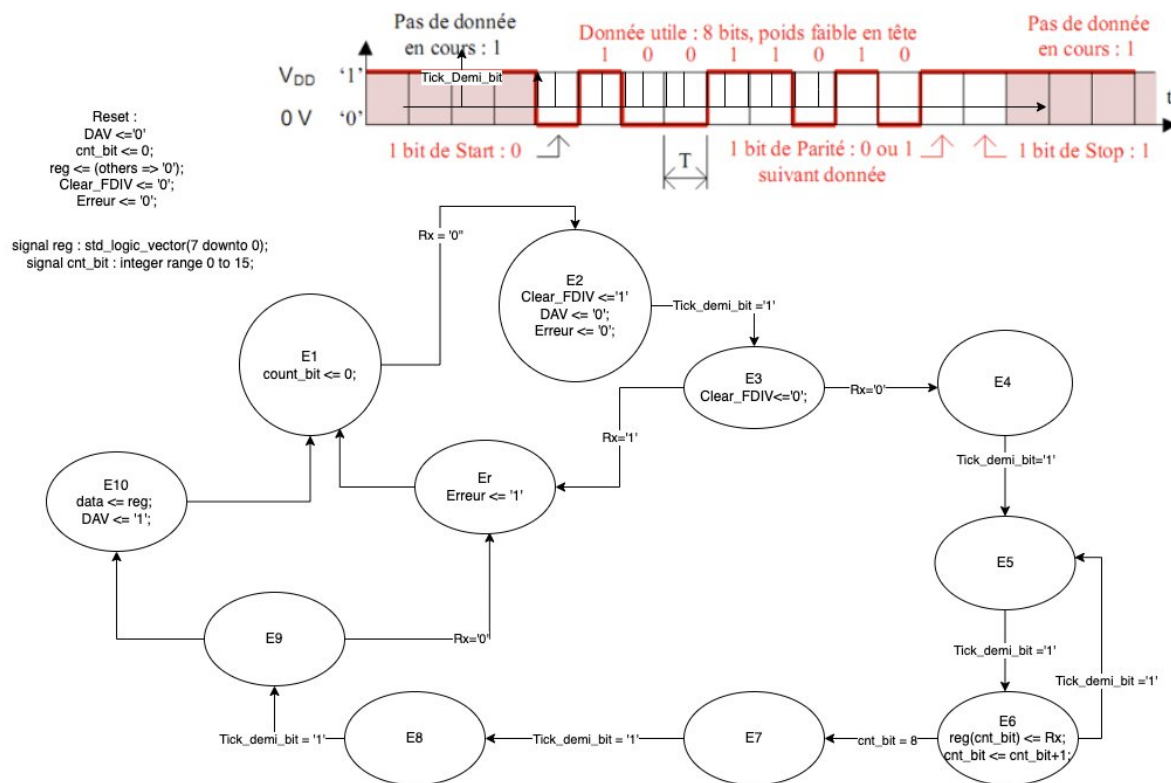
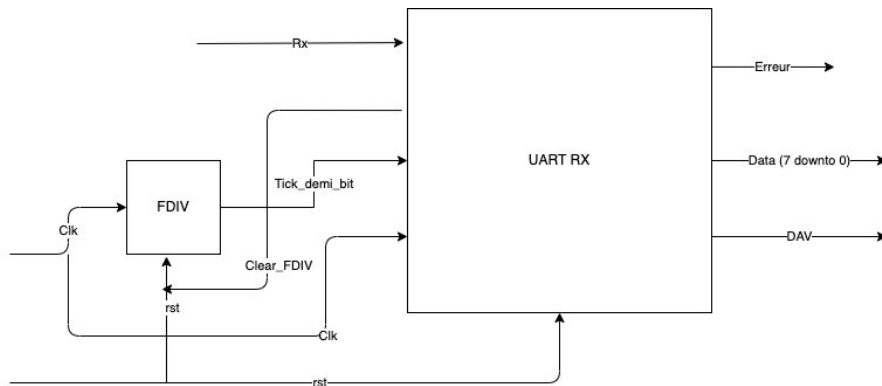


A FAIRE

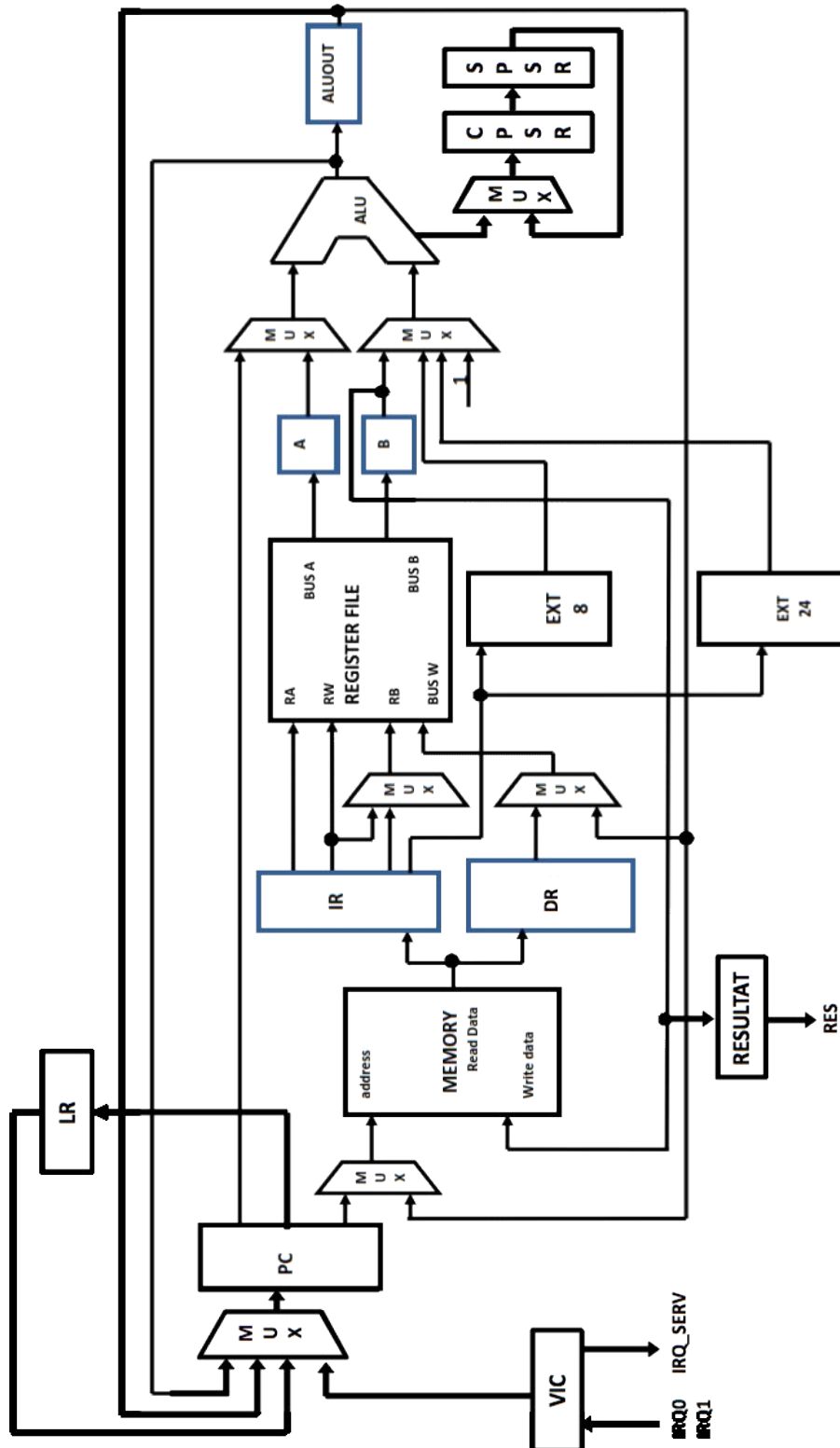
- 1) Créer une nouvelle source VHDL pour décrire la partie émission de l'UART
- 2) Simuler ce composant à l'aide d'un banc de test et un script de simulation.
- 3) Tester la partie opérative de l'UART sur la carte DE10-Lite en connectant la sortie Tx sur une pin du port GPIO (GPIO(0) par exemple), l'entrée data sur SW(7..0) et l'entrée Go sur KEY(1). Ensuite utiliser un convertisseur série/USB pour recevoir les caractères sur le terminal de votre PC (Puty sur Windows ou Screen sur Mac). Il faudra connecter la sortie Tx (GPIO(0)) sur le pin Rx (jaune) du convertisseur série/USB.
- 4) Rajouter un registre 32 bit UART\_Conf à votre périphérique. Celui-ci permet de recevoir l'octet à envoyer sur les bits 7 à 0. Chaque fois que vous écrivez sur le registre, vous devez mettre le Go à 1. Rajouter ce périphérique UART dans votre processeur. En utilisant une instruction STR à l'adresse 0x40 vous devez pouvoir écrire dans le registre UART\_Conf. Pour cela, il faut faire un décodage d'adresse qui permet d'écrire dans le registre que lorsque l'adresse est égale à 0x40.
- 5) Simuler le processeur avec ce nouveau périphérique en envoyant par exemple la valeur 0x31 ('1' en ASCII) à chaque fois que vous rentrez dans l'IRQ1.
- 6) Tester votre processeur avec le périphérique UART sur la carte DE10-Lite. Pour communiquer avec le PC, il faudra utiliser un convertisseur série/USB et utiliser un Terminal sur votre ordinateur (Puty sur Windows ou Screen sur Mac).
- 7) L'envoi d'un caractère prend un certain temps. Avec un baud rate de 115200, il faut attendre environ 87 us. Vous pouvez gérer cela par un mécanisme d'interruption. Lorsqu'un caractère a été envoyé vous pouvez générer une interruption et ainsi envoyer le prochain caractère dans le sous-programme d'interruption. Pour cela il faut générer un signal TxIrq dans votre UART à la fin de l'émission d'un caractère. Et il faudra également changer le VIC pour gérer cette nouvelle source d'interruption.
- 8) Modifier votre périphérique UART et le VIC afin de le gérer par interruption. Modifier le code assembleur à exécuter afin d'envoyer la chaîne de caractère HELLO WORLD dans le sous-programme d'interruption. Simuler le et tester le sur la carte FPGA.

## Partie réception de l'UART :

Faites la même chose pour la partie réception en vous aidant du schéma et de la MAE proposé ci-dessous.



## ANNEXE 1 – CHEMIN DE DONNEES et TABLEAU DES COMMANDES MULTICYCLE

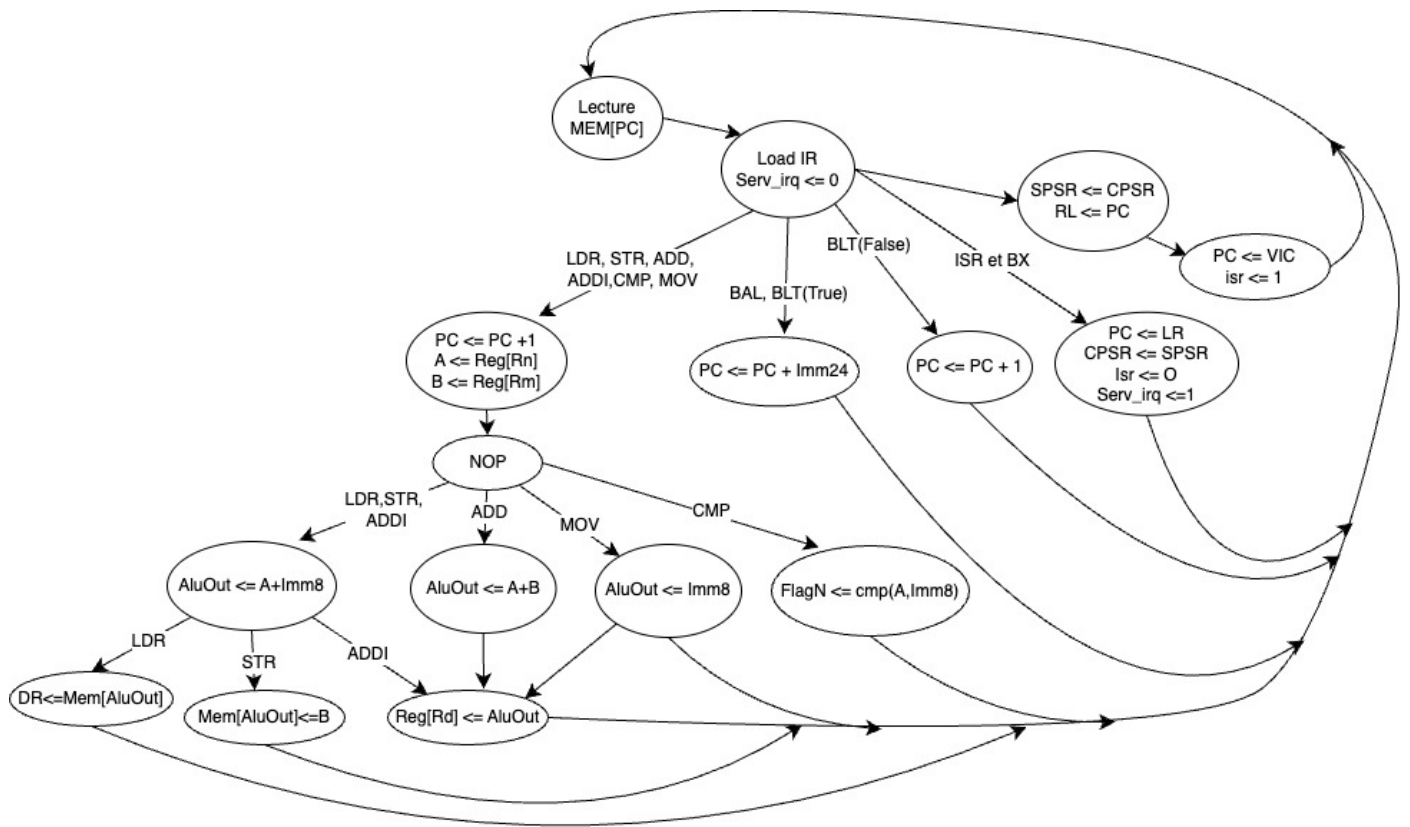


| ETAT<br>MAE | IRQ<br>Serv | PC<br>Sel | PC<br>WrEn | LR<br>WrEn | Adr<br>Sel | Mem<br>RdEn | Mem<br>WrEn | IR<br>WrEn | W<br>Sel | Reg<br>WrEn | ALU<br>SelA | ALU<br>SelB | ALU<br>OP | CPSR<br>Sel | CPSR<br>WrEn | SPSR<br>WrEn | Res<br>WrEn |
|-------------|-------------|-----------|------------|------------|------------|-------------|-------------|------------|----------|-------------|-------------|-------------|-----------|-------------|--------------|--------------|-------------|
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |

| ETAT<br>MAE | IRQ<br>Serv | PC<br>Sel | PC<br>WrEn | LR<br>WrEn | Adr<br>Sel | Mem<br>RdEn | Mem<br>WrEn | IR<br>WrEn | W<br>Sel | Reg<br>WrEn | ALU<br>SelA | ALU<br>SelB | ALU<br>OP | CPSR<br>Sel | CPSR<br>WrEn | SPSR<br>WrEn | Res<br>WrEn |
|-------------|-------------|-----------|------------|------------|------------|-------------|-------------|------------|----------|-------------|-------------|-------------|-----------|-------------|--------------|--------------|-------------|
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |
|             |             |           |            |            |            |             |             |            |          |             |             |             |           |             |              |              |             |



## ANNEXE 2 – GRAPHE D'ETATS MAE



## ANNEXES :

### Code assembleur de la partie 6 : DualPortRAM.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE IEEE.NUMERIC_STD.all;

ENTITY DualPortRAM IS
PORT (
    clock, rst: IN STD_LOGIC;
    data: IN STD_LOGIC_VECTOR (31 DOWNTO 0);
    q: out STD_LOGIC_VECTOR (31 DOWNTO 0);
    rdaddress, wraddress: IN STD_LOGIC_VECTOR (5 DOWNTO 0);
    rden, wren : in std_logic
);
END entity;

ARCHITECTURE rtl OF DualPortRAM IS
TYPE MEM IS ARRAY(0 TO 63) OF STD_LOGIC_VECTOR(31 DOWNTO 0);
SIGNAL ram_block: MEM;
BEGIN
PROCESS (clock,rst)
BEGIN
    IF rst = '1' then
        ram_block(0 ) <= x"E3A01020"; -- _main :   MOV R1,#0x20      ; --R1 <= 0x20
        ram_block(1 ) <= x"E3A02000"; --           MOV R2,#0        ; --R2 <= 0
        ram_block(2 ) <= x"E6110000"; -- _loop :   LDR R0,0(R1)     ; --R0 <= MEM[R1]
        ram_block(3 ) <= x"E0822000"; --           ADD R2,R2,R0     ; --R2 <= R2 + R0
        ram_block(4 ) <= x"E2811001"; --           ADD R1,R1,#1     ; --R1 <= R1 + 1
        ram_block(5 ) <= x"E351002A"; --           CMP R1,0x2A      ; --? R1 = 0x2A
        ram_block(6 ) <= x"BAFFFFFFC"; -- BLT loop    ; --branchement à _loop si R1 inferieur a
0x2A
        ram_block(7 ) <= x"E6012000"; --           STR R2,0(R1)     ; --MEM[R1] <= R2
        ram_block(8 ) <= x"EAffFFFF8"; --           BAL main        ; --branchement à _main
        -- ISR 0 : interruption 0
        --sauvegarde du contexte
        ram_block(9 ) <= x"E60F1000"; --           STR R1,0(R15)    ; --MEM[R15] <= R1
        ram_block(10) <= x"E28FF001"; --           ADD R15,R15,1    ; --R15 <= R15 + 1
        ram_block(11) <= x"E60F3000"; --           STR R3,0(R15)    ; --MEM[R15] <= R3
        --traitement
        ram_block(12) <= x"E3A03020"; --           MOV R3,0x20      ; --R3 <= 0x20
        ram_block(13) <= x"E6131000"; --           LDR R1,0(R3)     ; --R1 <= MEM[R3]
        ram_block(14) <= x"E2811001"; --           ADD R1,R1,1      ; --R1 <= R1 + 1
        ram_block(15) <= x"E6031000"; --           STR R1,0(R3)     ; --MEM[R3] <= R1
        -- restauration du contexte

```

```

ram_block(16) <= x"E61F3000"; --          LDR R3,0(R15)      ; --R3 <= MEM[R15]
ram_block(17) <= x"E28FF0FF"; --          ADD R15,R15,-1     ; --R15 <= R15 - 1
ram_block(18) <= x"E61F1000"; --          LDR R1,0(R15)      ; --R1 <= MEM[R15]
ram_block(19) <= x"EB000000"; -- BX              ; -- instruction de fin

d'interruption
ram_block(20) <= x"00000000";
-- ISR1 : interruption 1
--sauvegarde du contexte - R15 correspond au
pointeur de pile
ram_block(21) <= x"E60F4000"; --          STR R4,0(R15)      ; --MEM[R15] <= R4
ram_block(22) <= x"E28FF001"; --          ADD R15,R15,1      ; --R15 <= R15 + 1
ram_block(23) <= x"E60F5000"; --          STR R5,0(R15)      ; --MEM[R15] <= R5
--traitement
ram_block(24) <= x"E3A05020"; --          MOV R5,0x20        ; --R5 <= 0x20
ram_block(25) <= x"E6154000"; --          LDR R4,0(R5)        ; --R4 <= MEM[R5]
ram_block(26) <= x"E2844002"; --          ADD R4,R4,2        ; --R4 <= R1 + 2
ram_block(27) <= x"E6054000"; --          STR R4,0(R5)        ; --MEM[R5] <= R4
-- restauration du contexte
ram_block(28) <= x"E61F5000"; --          LDR R5,0(R15)      ; --R5 <= MEM[R15]
ram_block(29) <= x"E28FF0FF"; --          ADD R15,R15,-1     ; --R15 <= R15 - 1
ram_block(30) <= x"E61F4000"; --          LDR R4,0(R15)      ; --R4 <= MEM[R15]
ram_block(31) <= x"EB000000"; --          BX              ; -- instruction de fin

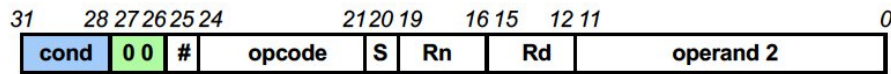
d'interruption
ram_block(32) <= x"00000001";
ram_block(33) <= x"00000002";
ram_block(34) <= x"00000003";
ram_block(35) <= x"00000004";
ram_block(36) <= x"00000005";
ram_block(37) <= x"00000006";
ram_block(38) <= x"00000007";
ram_block(39) <= x"00000008";
ram_block(40) <= x"00000009";
ram_block(41) <= x"0000000A";
ram_block(42 to 63) <= (others=> x"00000000");

elsif rising_edge(clock) THEN
    if (wren = '1') then
        ram_block(to_integer(unsigned(wrAddress))) <= Data;
    elsif (rden = '1') then
        q <= ram_block(to_integer(unsigned(rdAddress)));
    end if;
end if;

END PROCESS;
END architecture;
```

## Codage binaire d'une instruction de traitement :

### ■ Instructions de traitement (1)



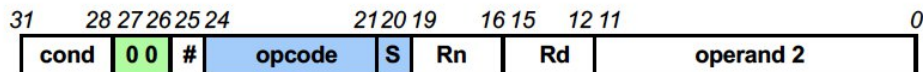
- *Cond*: l'instruction est exécutée si le registre d'état CPSR vérifie la condition spécifiée

| Asm   | Cond |
|-------|------|
| EQ    | 0000 |
| NE    | 0001 |
| CS/HS | 0010 |
| CC/LO | 0011 |
| MI    | 0100 |
| PL    | 0101 |

| Asm | Cond |
|-----|------|
| VS  | 0110 |
| VC  | 0111 |
| HI  | 1000 |
| LS  | 1001 |
| GE  | 1010 |
| LT  | 1011 |

| Asm | Cond |
|-----|------|
| GT  | 1100 |
| LE  | 1101 |
| AL  | 1110 |
| NV  | 1111 |

### ■ Instructions de traitement (2)



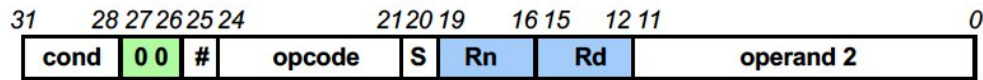
- *S = 1* : affecte CPSR
- *Opcode* : code de l'opération à effectuer

| Asm | Opcode |
|-----|--------|
| AND | 0000   |
| EOR | 0001   |
| SUB | 0010   |
| RSB | 0011   |
| ADD | 0100   |
| ADC | 0101   |

| Asm | Opcode |
|-----|--------|
| SBC | 0110   |
| RSC | 0111   |
| TST | 1000   |
| TEQ | 1001   |
| CMP | 1010   |
| CMN | 1011   |

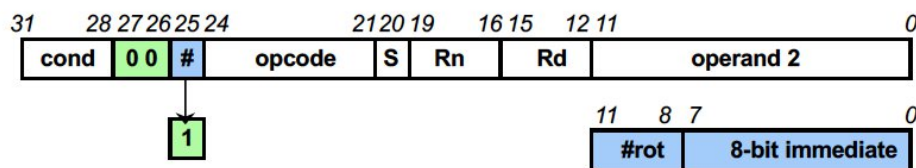
| Asm | Opcode |
|-----|--------|
| ORR | 1100   |
| MOV | 1101   |
| BIC | 1110   |
| MVN | 1111   |

## ■ Instructions de traitement (3)



- $Rd \rightarrow$  numéro du registre de destinations
  - 4 bits pour sélection parmi les 16 registres possibles
- $Rn \rightarrow$  numéro du registre qui sert de premier opérande
  - 4 bits pour sélection parmi les 16 registres possibles
- $operand2 \rightarrow$  relié à l'entrée B
  - possibilité de rotation/décalage

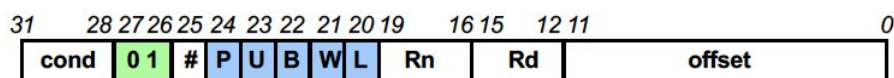
## ■ Instructions de traitement (4)



- Si le bit # est à 1,  $operand2$  est une valeur littérale sur 32 bits.
  - Une instruction est codée sur 32 bits, il n'est donc pas possible de coder n'importe quelle valeur littérale 32 bits.
  - Cette valeur est construite par rotation vers la droite d'une valeur sur 8 bits.

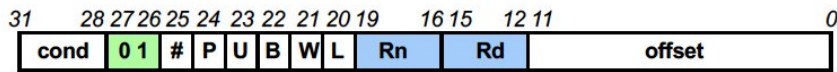


- Instructions de transfert
  - Lecture/écriture
  - Données accédées
    - Mots, demi-mots, octets
    - Signées/non signées
  - Mode d'accès (pré/post indexé, +/- offset, write back)
  - Transfert simple/multiple
  - Registre source/destination, liste de registres
  - Registre de base
  - Offset
  - Condition
    - Exécution conditionnelle d'un transfert
- Instructions de transfert de mots ou d'octets non-signés (LDR, STR, LDRB, STRB)

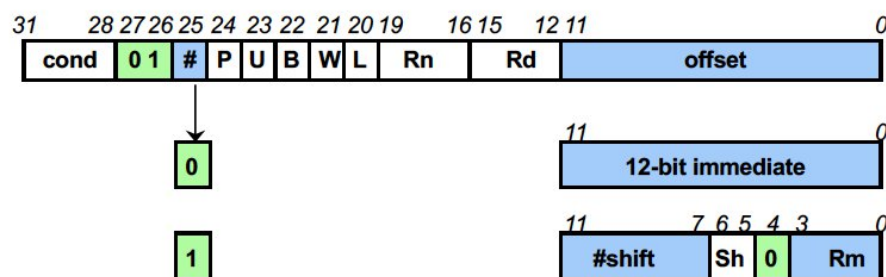


- *P* : pre/post index
  - 1 pré-indexé, 0 post-indexé
- *U* : up/down
  - 1 + offset, 0 -offset
- *B* : byte/word
  - M1 accès 8 bits, 0 accès 32 bits
- *W* : write-back
  - Si P=1, W=1 adressage pré-indexé automatique
- *L* : load/store
  - 1 load, 0 store

- Instructions de transfert de mots ou d'octets non-signés (LDR, STR, LDRB, STRB)



- $Rd \rightarrow$  registre source (si  $L=0$ , store) ou destination (si  $L=1$ , load)
- $Rn \rightarrow$  registre de base
- Instructions de transfert de mots ou d'octets non-signés (LDR, STR, LDRB, STRB)



- Offset: soit un littéral non signé sur 12 bits, soit un registre d'index ( $Rm$ ) éventuellement décalé sur un nombre constant de bits ( $\#shift$ )

## Instructions de branchement relatif

- Branch (B)
- Branch and Link (BL)



- Offset : déplacement signé sur 24 bits
- L : Link (0 branch, 1 branch and link)
- Effets:
  - B:  $PC \leftarrow PC + \text{offset}$
  - BL:  $r14 \leftarrow PC - 4$ ;  $PC = PC + \text{offset}$
  - r14 : Link Register (contient l'adresse de retour)

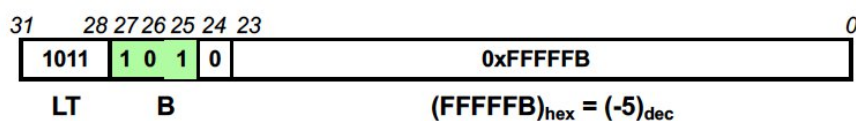
## Instructions de branchement relatif

- Exemple traduction d'une boucle *for*
- Utilisation de labels en langage d'assemblage: le déplacement est calculé automatiquement par l'assembleur

```

                                MOV      r4, #0           @ tmp=0
                                MOV      r5, #0           @ i=0
loop:                          ADD      r4, r4, r5       @ tmp+=i
                                ADD      r5, r5, #1       @ i++
                                CMP      r5, #5
                                BLT      loop             @ i<5 : réitérer
                                ...

```



Représentation binaire: BAFFFFFFB